

A MINI PROJECT REPORT ON
CUSTOMER CHURN PREDICTION USING
REGRESSION ANALYSIS

Submitted in partial fulfilment of the requirement for the award of degree

BACHELOR OF TECHNOLOGY

IN

INFORMATION TECHNOLOGY

SUBMITTED BY

NAVYA SREE BATTA

22671A1235

Under the Supervision of

Mr. U. KARTHIK

Assistant Professor



DEPARTMENT OF INFORMATION TECHNOLOGY
J.B INSTITUTE OF ENGINEERING & TECHNOLOGY

(UGC AUTONOMOUS)

(Accredited by NAAC & NBA, Approved by AICTE & Permanently affiliated to JNTUH)

Yenkapally, Moinabad mandal, R.R. Dist-75(TG) (2025 – 2026)

J.B. INSTITUTE OF ENGINEERING & TECHNOLOGY

(UGC AUTONOMOUS)

(Accredited by NAAC & NBA, Approved by AICTE & Permanently affiliated to JNTUH)

Yenkapally, Moinabad mandal, R.R. Dist-75(TG)

DEPARTMENT OF INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the Mini Project entitled **CUSTOMER CHURN PREDICTION USING REGRESSION ANALYSIS** submitted by **NAVYA SREE BATTA** bearing Roll Number: **22671A1235** to the Department of Information Technology, J.B Institute of Engineering & Technology, in accordance with Jawaharlal Nehru Technological University regulations as partial fulfillment required for successful completion of Bachelor of Technology is a record of Bonafide work carried out during the academic year 2025-2026.

INTERNAL GUIDE

Mr. U. KARTHIK

Assistant Professor

HEAD OF THE DEPARTMENT

Dr. A. RAMESH BABU

Associate Professor

J.B. INSTITUTE OF ENGINEERING & TECHNOLOGY

(UGC AUTONOMOUS)

(Accredited by NAAC & NBA, Approved by AICTE & Permanently affiliated to JNTUH)

Yenkapally, Moinabad mandal, R.R. Dist-75(TG)

DEPARTMENT OF INFORMATION TECHNOLOGY



DECLARATION

I hereby declare that the Mini Project report entitled **CUSTOMER CHURN PREDICTION USING REGRESSION ANALYSIS** carried out under the guidance of, **Mr. U. KARTHIK, Assistant Professor** is submitted in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Information Technology. This is a record of Bonafide work carried out by us and the results embodied in this project have not been reproduced or copied from any source. The results embodied in this project report have not been submitted to any other university or institute for the award of any other degree or diploma.

Date:

SIGNATURE OF THE STUDENT

NAVYA SREEE BATTA

22671A1235

ACKNOWLEDGEMENT

At outset I express my gratitude to almighty lord for showering his grace and blessings upon me to complete this Mini Project. Although my name on the cover of this book, many people have contributed in some form or the other to this project development. I could not have done this project without the assistance or support of each of the following.

First of all, I am highly indebted to **Dr. P.C. KRISHNAMACHARY**, Principal for giving us the permission to carry out this Mini Project.

I would like to thank **Dr. A. RAMESH BABU**, Associate Professor & Head of the Department of Information Technology, for being moral support throughout the period of the study in the Department.

I am grateful to **Mr. U. KARTHIK**, Assistant Professor Information Technology for his valuable suggestions and guidance given by him during the execution of this Project work.

I would like to thank the Teaching and non-Teaching staff of the Department of Information for sharing their knowledge with me.

.

NAVYA SREE BATT

22671A1235

ABSTRACT

The project lies in the field of Data Science and Machine Learning, focusing on Customer Relationship Management (CRM). It uses predictive analytics to identify customers likely to churn and supports data-driven retention strategies.

Customer churn reduces revenue and increases acquisition costs. Since retaining existing customers is more cost-effective, predicting churn is vital to help businesses take proactive steps to maintain customer loyalty. Traditional systems rely on manual analysis and fixed business rules, which are time-consuming, less accurate, and unable to handle large datasets or capture complex customer behaviours.

The proposed system applies Logistic Regression to build a machine learning-based churn prediction model. It analyses historical and behavioural data to estimate churn probability and features a Flask-based web interface for real-time prediction and analysis.

The implementation involves data preprocessing (handling missing values, encoding, and normalization), followed by feature engineering to identify key churn indicators. The Logistic Regression model is then trained and evaluated using accuracy and F1-score metrics. Finally, the model is deployed with Flask, and a web interface is created for real-time churn prediction and visualization.

TABLE OF CONTENTS

TITLE	Pg No
Certificate.....	i
Declaration	ii
Acknowledgement	iii
Abstract.....	iv
Table of Content.....	v
List of Figures.....	vii
List of Tables	viii
Abbreviations.....	ix
1. Introduction	1
1.1 Introduction	1
1.2 Problem Statement	1
1.3 Objectives.....	2
1.4 Motivation	3
1.5 Scope and Challenges.....	3
2. Literature Review.....	5
2.1 Existing Systems	5
2.2 Areas for Improvement	6
3. Proposed System and Methodology	7
3.1 Functional Requirement	7
3.2 Non-Functional Requirements	8
3.3 Hardware Requirements	9
3.4 Software Requirements.....	10
3.5 Data Collection Process	10
3.6 Deliverables and Beneficiaries	11
3.7 Algorithm	11
3.8 Methodology	12

4. System Design	13
4.1 ER Diagrams	13
4.2 System Architecture	14
4.3 Use Case Diagram.....	15
4.4 Class Diagram	16
4.5 Activity Diagram.....	18
4.6 Sequence Diagram	19
5. System Implementation	21
5.1 Source Code (google.colab).....	21
5.2 Index.html	28
5.3 App.py	31
5.4 Create_model.py.....	32
6. System Testing.....	33
6.1 Unit Testing.....	34
6.2 Integration Testing	36
6.3 Functional Testing.....	38
6.4 System Testing	40
6.5 White Box Testing	42
6.6 Black Box Testing.....	44
7. Results and Discussion	46
7.1 Landing Page & Navigation.....	46
7.2 Prediction Testing Case 1... ..	44
7.3 Prediction Testing Case 2.....	45
7.4 Discussion	50
8. Conclusion and Future Work.....	52
8.1 Conclusion.....	52
8.2 Future work	53
References	55

LIST OF FIGURES

Figure No.	Title of the Figures	Page No.
Figure 4.1	ER Diagrams	13
Figure 4.2	System Architecture	14
Figure 4.3	Use Case Diagram.....	15
Figure 4.4	Class Diagram	16
Figure 4.5	Activity Diagram.....	18
Figure 4.6	Sequence Diagram	19
Figure 7.1	Landing Page.....	46
Figure 7.2	Real-Time Churn Prediction (High Risk Customer)	48
Figure 7.3	Real-Time Churn Prediction (Low Churn Risk Customer).....	49

LIST OF TABLES

Table No.	Title of the Table	Page No.
Table 3.3	Hardware Requirements.....	9
Table 3.4	Software Requirements	10
Table 6.1	Unit Testing.....	35
Table 6.2	Integration Testing	37
Table 6.3	Functional Testing.....	39
Table 6.4	System Testing	41
Table 6.5	White Box Testing	43
Table 6.6	Black Box Testing.....	45

LIST OF ABBREVIATIONS

AI	—	Artificial Intelligence
ML	—	Machine Learning
EDA	—	Exploratory Data Analysis
CRM	—	Customer Relationship Management
ETL	—	Extract, Transform, Load
DV	—	Dependent Variable
IV	—	Independent Variable
LR	—	Logistic Regression
RMSE	—	Root Mean Squared Error
ACC	—	Accuracy
PR	—	Precision
REC	—	Recall
F1	—	F1 Score
KPI	—	Key Performance Indicator
SQL	—	Structured Query Language
ROC	—	Receiver Operating Characteristic
MSE	—	Mean Squared Error
DBMS	—	Database Management System

Chapter 1

INTRODUCTION

1.1 Introduction

Customer churn prediction is an important technique used by businesses to identify customers who are likely to discontinue using their products or services. In today's competitive market, retaining existing customers is more cost-effective than acquiring new ones. Therefore, predicting churn in advance helps companies take proactive steps such as offering discounts, improving service quality, or personalizing customer experiences to reduce churn rates.

In this project, a Customer Churn Prediction Model is developed using Regression Analysis, a statistical method that examines the relationship between customer characteristics and their likelihood of leaving. By analyzing factors such as usage patterns, customer demographics, account details, and service feedback, the model estimates the probability of churn. Regression analysis is chosen because it provides interpretability, simplicity, and efficiency in identifying the most influential factors affecting customer decisions.

The model helps businesses gain insights into customer behavior, improve decision-making, and enhance customer retention strategies. Overall, this approach demonstrates how data-driven techniques can be applied to predict and minimize customer churn effectively.

1.2 Problem Statement

In competitive business environments, customer retention has become a major challenge for organizations. Many companies experience high customer turnover, which directly impacts revenue, growth, and brand loyalty. However, identifying which customers are likely to leave before they actually do is a complex task, as it depends on multiple factors such as customer behavior, service quality, satisfaction levels, and pricing.

The main problem addressed in this project is the prediction of customer churn using regression analysis. The goal is to analyze customer data, identify key factors influencing churn, and develop a model that can accurately predict the likelihood of a customer leaving. By doing so, businesses can take timely actions to retain valuable customers, reduce losses, and improve overall customer satisfaction.

This project aims to provide a reliable, data-driven approach to churn prediction, helping organizations make informed decisions and strengthen their customer relationship management strategies.

1.3 Objective

The main objective of this project is to develop a Customer Churn Prediction Model using Regression Analysis that can identify customers who are likely to discontinue their services. The model aims to analyze customer data and uncover patterns that contribute to churn, enabling businesses to take preventive measures in advance.

The specific objectives are:

- To collect and preprocess customer data from various sources such as CRM systems and transaction records.
- To identify and select significant features that influence customer churn.
- To apply regression analysis techniques to model the relationship between customer attributes and churn probability.
- To evaluate the model's accuracy and reliability using appropriate performance metrics.
- To provide actionable insights that help organizations improve customer retention strategies and reduce churn rates.

Overall, the objective is to build a predictive framework that supports data-driven decision-making and enhances business sustainability through effective customer retention.

1.4 Motivation

In today's competitive market, retaining customers is far more profitable than constantly acquiring new ones. Studies show that even a small reduction in customer churn can lead to significant increases in a company's long-term profits. This makes churn prediction an essential task for businesses that aim to sustain growth and maintain customer loyalty.

The motivation behind this project is to use data analytics and regression techniques to understand the hidden factors that drive customers to leave a service. By identifying these patterns early, businesses can design targeted retention strategies, offer personalized services, and improve customer satisfaction.

Furthermore, the project highlights how machine learning and statistical models can be practically applied to solve real-world business problems. It encourages the use of data-driven decision-making to support customer relationship management and demonstrates how predictive models can help organizations move from reactive to proactive strategies in managing customer churn.

1.5 Scope and Challenges

Scope:

The scope of this project focuses on developing a Customer Churn Prediction Model using Regression Analysis to identify customers who are likely to discontinue a service. The project involves collecting customer-related data such as demographics, account information, and usage behavior, followed by data preprocessing, feature selection, and model development. The regression model will estimate the probability of churn and help businesses understand the key factors influencing customer decisions. The insights gained can be used by marketing and customer service teams to design personalized retention strategies, improve customer satisfaction, and reduce churn rates. This model can be applied across various industries such as telecommunications, banking, and e-commerce where customer retention is critical.

Challenges:

Developing an accurate churn prediction model comes with several challenges:

- **Data Quality Issues:** Customer data may contain missing, inconsistent, or incomplete records that affect model accuracy.
- **Feature Selection:** Identifying the most relevant variables that strongly influence churn is complex and time-consuming.
- **Class Imbalance:** The number of churned customers is often much smaller than non-churned ones, leading to biased predictions.
- **Model Interpretability:** Ensuring that regression results are understandable to business users while maintaining statistical accuracy.
- **Dynamic Customer Behavior:** Customer preferences and behaviors change over time, requiring regular model updates.
- **Data Privacy:** Handling customer information responsibly while complying with data protection regulations.

Overall, the project's scope emphasizes building a predictive model that is both reliable and interpretable, while the challenges highlight the practical difficulties faced during data analysis and model development.

Chapter-2

LITERATURE REVIEW

2.1 Existing Systems

Logistic Regression and Decision Trees: Rani et al. [1] analyzed customer churn prediction in telecom using decision trees and logistic regression. Their study showed that logistic regression achieved better accuracy and provided interpretable results for predicting customer churn based on call duration, complaints, and service usage patterns.

Random Forest Approach: Xie et al. [2] proposed an improved balanced random forests method to address class imbalance in churn datasets. The balanced approach improved prediction accuracy and better identified minority class instances, demonstrating that ensemble methods effectively handle imbalanced data.

Boosting Techniques: Zhao et al. [3] developed a churn prediction model using gradient boosting algorithms for the telecom industry. The boosting method outperformed traditional single-classifier approaches by sequentially building weak learners focused on misclassified instances.

Big Data Analytics: Hadden [4] presented a logistic regression-based model emphasizing big data analysis for telecommunications. The study demonstrated scalability in handling large-scale customer datasets and identified key churn indicators for business decision-making.

Comparative Machine Learning Analysis: Simsek et al. [5] conducted a comprehensive comparison of various machine learning methods including logistic regression, SVM, naive Bayes, and decision trees. Results showed that ensemble methods generally outperformed individual classifiers.

Deep Learning Approach: Agrawal et al. [6] utilized neural networks to analyze behavioral patterns from customer interactions. The deep learning approach captured non-linear relationships and hidden patterns, achieving superior accuracy compared to traditional methods.

Novel Optimization Techniques: Aljunid et al. [7] introduced innovative feature engineering and hybrid modeling approaches, emphasizing domain-specific feature selection and model optimization for improved prediction performance.

2.2 Areas for Improvement

Class Imbalance: Studies [1], [2] revealed that class imbalance remains challenging. Advanced techniques like SMOTE, cost-sensitive learning, and hybrid sampling methods need implementation for better minority class prediction.

Feature Engineering: Research [3], [7] highlighted the need for automated feature extraction, dimensionality reduction, and advanced selection methods like recursive feature elimination to improve model performance.

Real-Time Prediction: Study [4] demonstrated scalability needs, but real-time churn prediction requires streaming analytics pipelines and online learning algorithms for instantaneous predictions.

Model Interpretability: While [5], [6] showed effectiveness of complex models, there is a need for explainable AI techniques like SHAP and LIME to provide transparent insights for business stakeholders.

Hybrid Models: Studies [2], [3], [7] explored different algorithms individually. Developing sophisticated hybrid models combining multiple approaches through stacking and blending could enhance performance.

Customer Segmentation: Studies [1], [3], [4], [5] used global prediction models. Segment-specific models based on demographics and usage patterns could yield more accurate predictions.

Temporal Analysis: Research [3], [6] touched upon behavioral patterns, but comprehensive temporal analysis using LSTM or RNNs could better capture time-dependent customer behavior and lifecycle stages.

Chapter-3

PROPOSED SYSTEM AND METHODOLOGY

3.1 Functional Requirements

The functional requirements define the essential features that the Churn prediction model must perform:

- **Data Collection**

The system should collect customer data from multiple sources such as CRM systems, billing records, and customer support logs to ensure comprehensive input for churn analysis.

- **Data Preprocessing**

It should handle missing values, remove duplicates, and normalize data to prepare it for regression-based analysis.

- **Feature Selection**

The model must identify and select the most relevant attributes (like tenure, monthly charges, contract type, etc.) that significantly influence customer churn.

- **Data Splitting**

The system should split the dataset into training and testing sets to evaluate model performance accurately.

- **Model Training**

It should train a regression model (such as Logistic Regression) using the training data to learn the relationship between customer features and churn probability.

- **Model Testing and Validation**

The system should test the model using unseen test data and validate the performance through metrics like accuracy, precision, recall, and F1-score.

- **Prediction of Churn Probability**

The model should predict the probability of churn for each customer, indicating how likely they are to leave the service.

- **Result Visualization**

The system must display results in graphical or tabular form, such as churn probability charts, confusion matrices, or trend plots.

- **Model Optimization**

It should allow fine-tuning of model parameters (e.g., regularization strength) to improve accuracy and minimize errors.

- **User Interface for Prediction**

The system should provide a simple interface where users can input new customer data and instantly get churn prediction results.

- **Report Generation**

The system must generate a detailed report showing predicted churners, key influencing factors, and performance metrics.

- **Data Storage and Retrieval**

It should maintain a secure database for storing historical data, model parameters, and prediction results for future reference and analysis.

3.2 Non-Functional Requirements

These requirements define the quality attributes of the system:

- **Performance**

The system should process large customer datasets efficiently and generate churn predictions within minimal response time.

- **Scalability**

It must be capable of handling increasing data volume and user requests without affecting system performance.

- **Accuracy**

The model should maintain a high prediction accuracy, ensuring reliable identification of potential churners.

- **Usability**

The interface should be user-friendly, allowing even non-technical users to input data and interpret churn results easily.

- **Reliability**

The system should perform consistently without failure, ensuring dependable churn predictions during continuous use.

- **Security**

All customer data and prediction results must be stored and transmitted securely to protect sensitive information.

- **Maintainability**

The model and codebase should be easy to update or modify to include new features, datasets, or improved algorithms.

- **Portability**

The system should be deployable across multiple platforms such as Windows, Linux, or cloud environments.

- **Data Integrity**

The system must ensure that customer data remains accurate, consistent, and unaltered throughout the processing stages.

- **Availability**

The system should be accessible at all times to authorized users for real-time prediction and analysis.

3.3 Hardware Requirements

Component	Minimum Requirement
Processor (CPU)	Intel Core i3 or equivalent
RAM	4 GB
Hard Disk	250 GB HDD
Monitor	15-inch display
Graphics Card	Integrated graphics
Keyboard & Mouse	Standard input devices
Network Adapter	Ethernet / Wi-Fi (802.11n)
Operating System	Windows 10 / Linux (64-bit)
Power Supply	230V AC, 50Hz

Table 3.3: Hardware Requirements

3.4 Software Requirements

Software Component	Minimum Requirement
Operating System	Windows 10 / Linux (64-bit)
Programming Language	Python 3.8 or above
Libraries/Packages	NumPy, Pandas, Scikit-learn, Matplotlib, Flask
Database	MySQL / SQLite
IDE / Code Editor	Jupyter Notebook / VS Code / PyCharm
Web Browser	Google Chrome / Mozilla Firefox
Visualization Tool	Power BI / Tableau (optional)
Version Control	Git / GitHub (optional)

Table 3.4 Software Requirements

3.5 Data Collection Process

- **Define Objective** – Identify the goal of churn prediction and define the target variable (churn = 1, no churn = 0).
- **Identify Data Sources** – Collect data from CRM systems, billing records, usage logs, and customer support databases.
- **Select Attributes** – Choose important features such as tenure, monthly charges, contract type, payment method, and service usage.
- **Data Extraction** – Extract required data using SQL queries or APIs and store it in a structured format like CSV or database tables.
- **Data Quality Check** – Remove duplicates, handle missing values, and verify consistency of collected data.
- **Labelling** – Assign churn labels to customers based on whether they discontinued the service during a specific period.
- **Ensure Privacy and Security** – Protect sensitive customer information by applying encryption and access control.

- **Data Storage and Documentation** – Store collected data securely in a database or cloud storage for further preprocessing and model training, maintain proper records of data sources.

3.6 Deliverables and Beneficiaries

The main deliverables of the Customer Churn Prediction Model using Regression Analysis include a fully trained machine learning model that can accurately predict whether a customer is likely to churn or remain loyal. A preprocessed dataset will be prepared to ensure data cleanliness and consistency for model training. The project will also deliver a detailed performance report containing evaluation metrics such as accuracy, precision, recall, and F1-score to measure model effectiveness. Additionally, a visualization dashboard will be developed to display churn trends, prediction results, and customer insights in an easy-to-understand manner. A user-friendly web interface will be provided for users to input customer data and obtain real-time churn predictions. The project documentation will include details of objectives, methodology, algorithms, and results, while a source code repository will be maintained for transparency and future use.

The key beneficiaries of this model are telecom and subscription-based companies that can use it to identify potential churners and take preventive measures to retain them. Marketing teams can design personalized offers and retention campaigns based on churn insights, while customer support departments can improve service quality by understanding reasons behind customer dissatisfaction. Business analysts and decision-makers can use the model's insights to develop effective business strategies and reduce revenue loss. Furthermore, researchers and students can benefit from the project as a reference for future studies or advanced churn prediction research.

3.7 Algorithm

Algorithm for Customer Churn Prediction using Regression Analysis:

The process begins with data collection, where customer information such as demographics, service usage, billing details, and support interactions is gathered from

relevant databases. Next, data preprocessing is performed, which includes handling missing values, removing duplicates, encoding categorical variables, and normalizing numerical features. After preprocessing, feature selection identifies the most relevant attributes that influence customer churn. The dataset is then split into training and testing sets to ensure accurate evaluation of the model. A regression model, typically logistic regression, is trained using the training data to learn the relationship between customer features and churn probability. Once trained, the model is tested and validated on the unseen test set using performance metrics such as accuracy, precision, recall, and F1-score. Finally, the model is used to predict churn probabilities for new or existing customers, and the results are visualized through dashboards or reports for business decision-making.

3.8 Methodology

The methodology for predicting customer churn begins with data collection, where relevant customer information such as demographics, service usage patterns, billing details, and support interactions is gathered from CRM systems, billing databases, and other internal sources. This is followed by data preprocessing, which includes handling missing values, removing duplicates, encoding categorical variables, and normalizing numerical features to make the data suitable for regression analysis. The next step is feature selection, where the most significant attributes affecting churn, such as tenure, contract type, payment method, and monthly charges, are identified. The prepared dataset is then split into training and testing sets to allow proper model evaluation. A regression model, typically logistic regression, is trained on the training data to learn the relationship between customer features and churn probability. The model is then tested and validated using the testing set with performance metrics like accuracy, precision, recall, and F1-score. Finally, the trained model is used to predict churn probabilities for current or new customers, and the results are visualized through dashboards or reports to assist business stakeholders in making informed decisions for customer retention.

Chapter-4

SYSTEM DESIGN

4.1 ER Diagrams

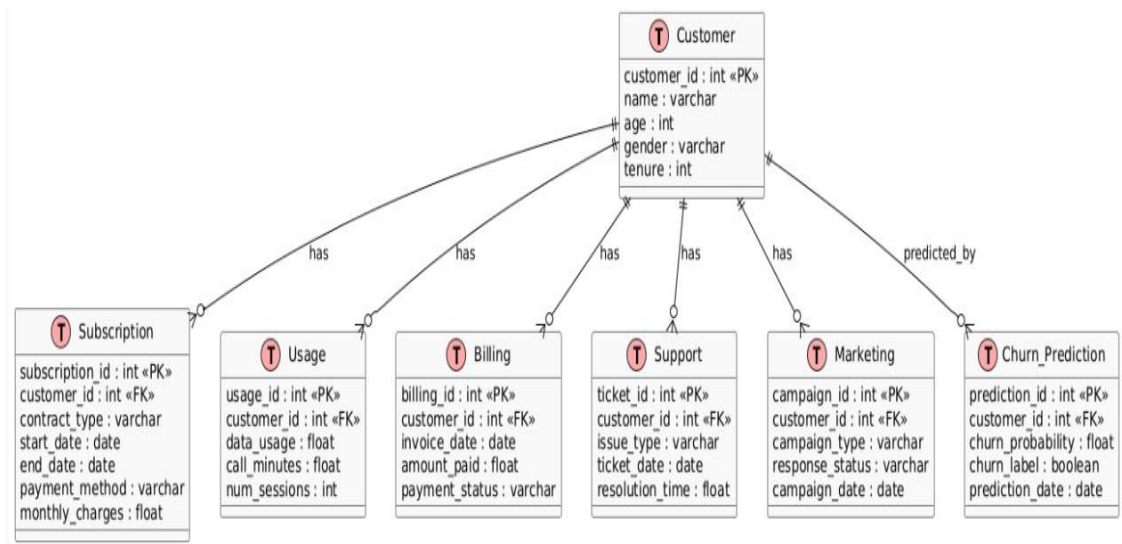


Figure 4.1: ER Diagrams

This ER diagram shows a Customer Churn Prediction System with the following structure:

Central Entity: Customer

- Contains: customer_id (PK), name, age, gender, tenure
- Acts as the hub connecting all other entities

Six Connected Entities:

- Subscription - Contract details, payment method, monthly charges
- Usage - Data usage, call minutes, number of sessions
- Billing - Invoice dates, payment amounts and status
- Support - Customer service tickets and resolution time
- Marketing - Campaign interactions and response status
- Churn Prediction - Predicted churn probability and actual churn label

Relationships:

- All entities have a "has" relationship with Customer (1:Many)
- Churn_Prediction has a "predicted_by" relationship

4.2 System Architecture

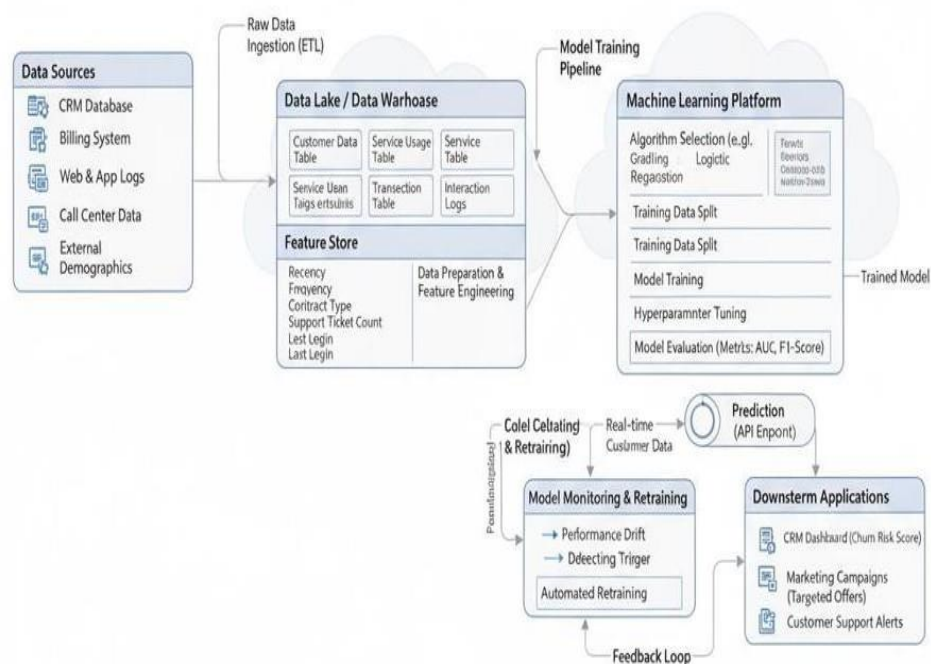


Figure 4.2: System Architecture

This System Architecture Diagram shows how the churn prediction model works:

- Data Sources → CRM, Billing, Logs, Call Center, Demographics
- Etl & Storage → Data Lake stores tables + Feature Store creates predictive features (recency, frequency, tenure)
- Model Training → ML Platform trains regression models (Logistic/Gradient), tunes hyperparameters, evaluates performance
- Deployment → Trained model → Real-time Prediction API

- Monitoring → Tracks model drift → Triggers automated retraining
- Applications → Predictions used in CRM dashboards, marketing campaigns, support alerts

4.3 Use Case Diagram

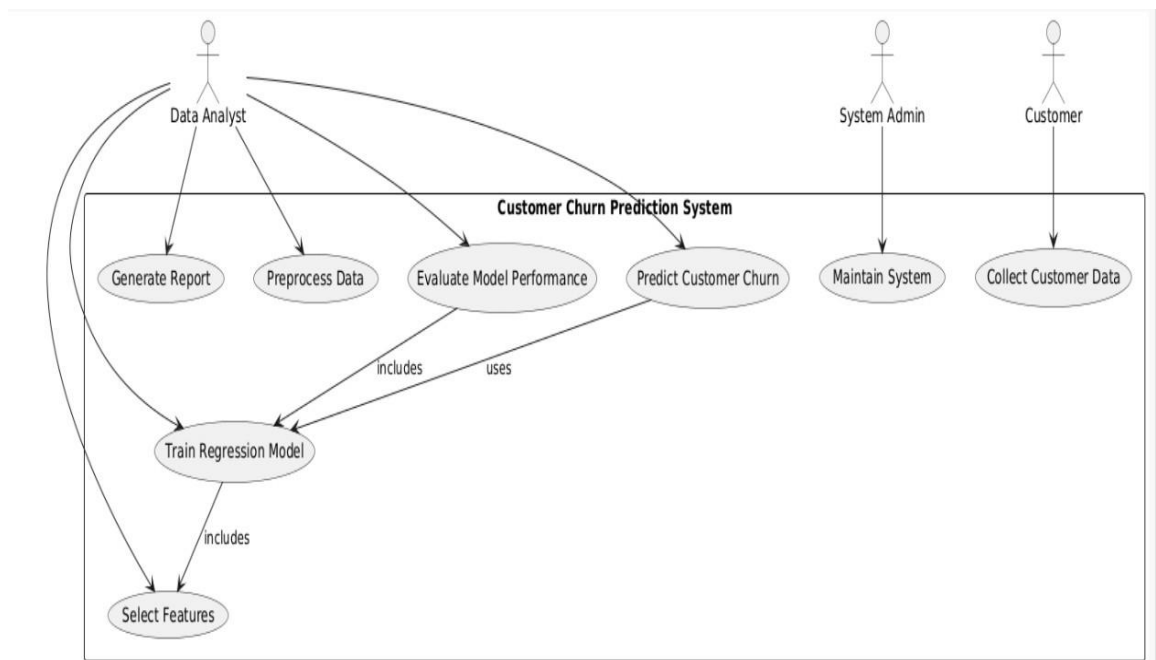


Figure 4.3: Use Case Diagram

This diagram shows a Customer Churn Prediction System with three users: Data Analyst, System Admin, and Customer.

The Data Analyst performs the core tasks of pre-processing data, training regression models with feature selection, evaluating performance, making churn predictions, and generating reports. The System Admin maintains the system while the Customer provides the raw data.

The diagram illustrates dependencies where model training includes feature selection, evaluation requires a trained model, and predictions use the trained model to forecast customer churn.

4.4 Class Diagram

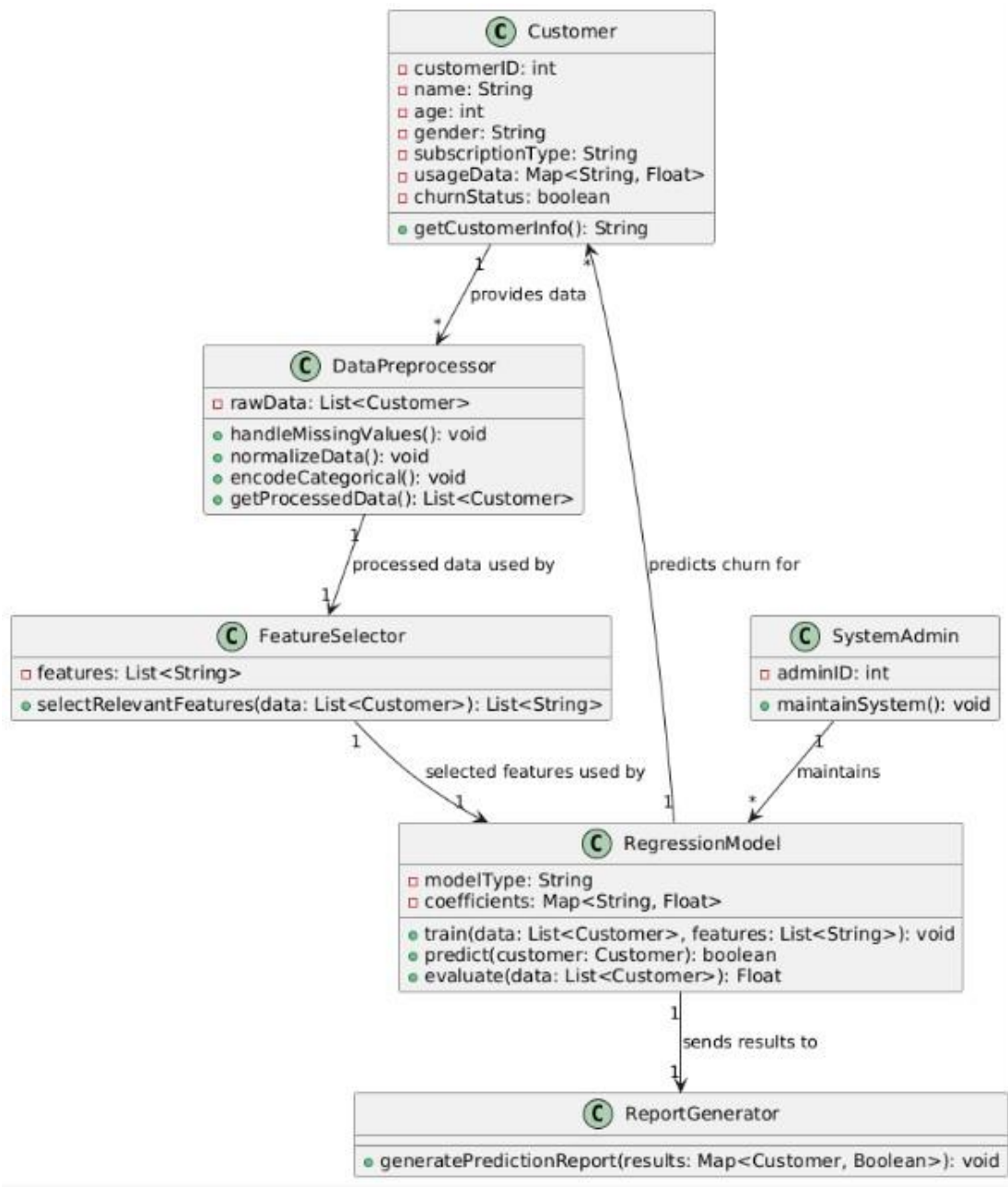


Figure 4.4: Class Diagram

Class diagram shows six classes working together:

Customer stores customer data (ID, name, age, subscription type, usage data, churn status) and provides it to DataPreprocessor, which cleans the data and handles missing values.

FeatureSelector selects relevant features from the processed data and feeds them to RegressionModel, which trains on the data, predicts churn probability for customers, and evaluates model performance.

RegressionModel sends prediction results to ReportGenerator, which creates prediction reports showing which customers will churn.

SystemAdmin maintains the entire system.

Flow: Raw customer data → preprocessed → features selected → model trained → predictions made → reports generated

4.5 Activity Diagram

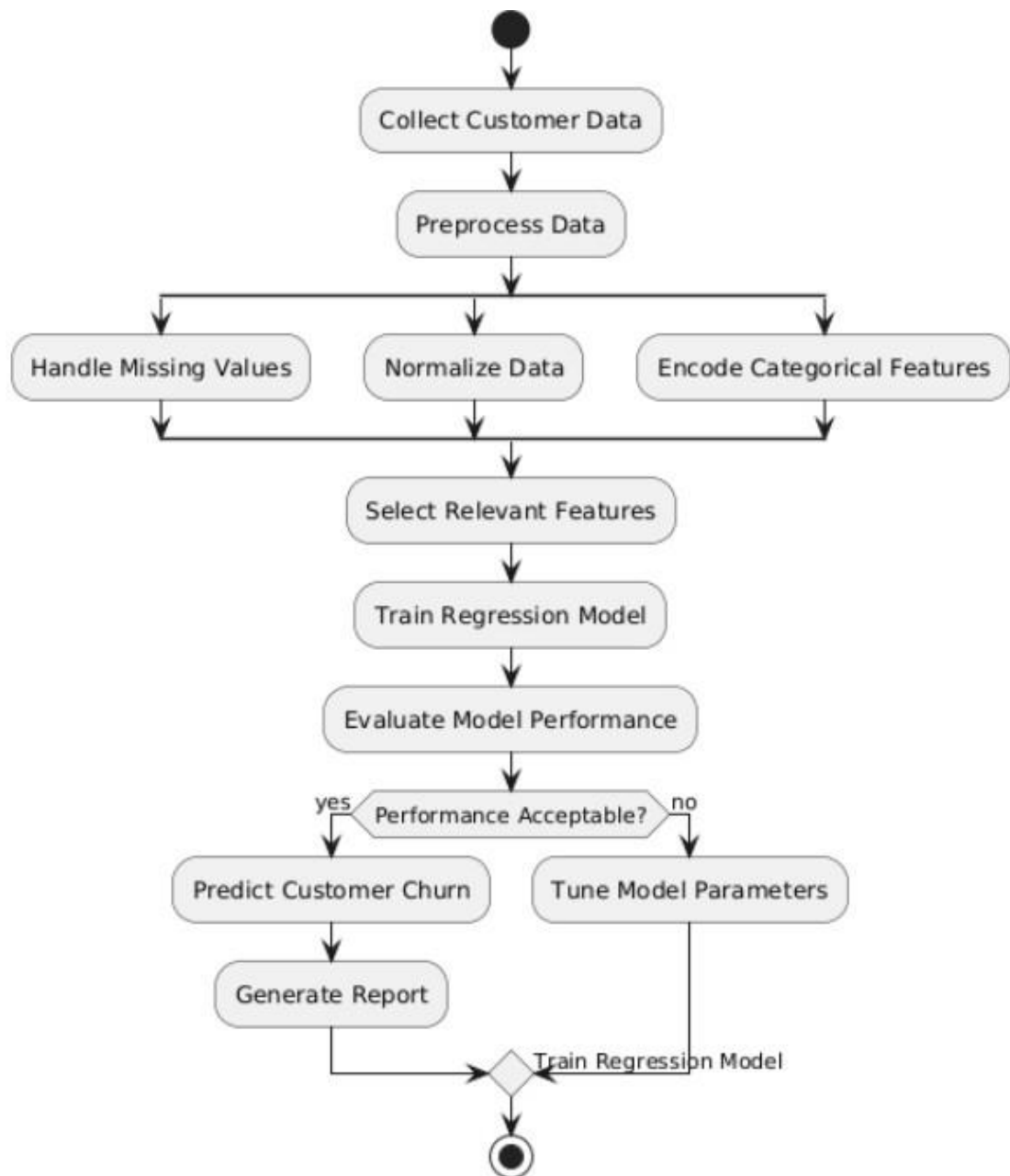


Figure 4.5: Activity Diagram

- **Process flow:** Collect customer data → Preprocess data in three parallel steps (handle missing values, normalize data, encode categorical features) → Select relevant features → Train regression model → Evaluate performance.
- **Decision point:** If performance is acceptable, predict customer churn and generate report. If not acceptable, tune model parameters and retrain until performance is satisfactory.

- **Purpose:** Shows the complete step-by-step process from raw data collection to final churn prediction report with iterative model improvement.

4.6 Sequence Diagram

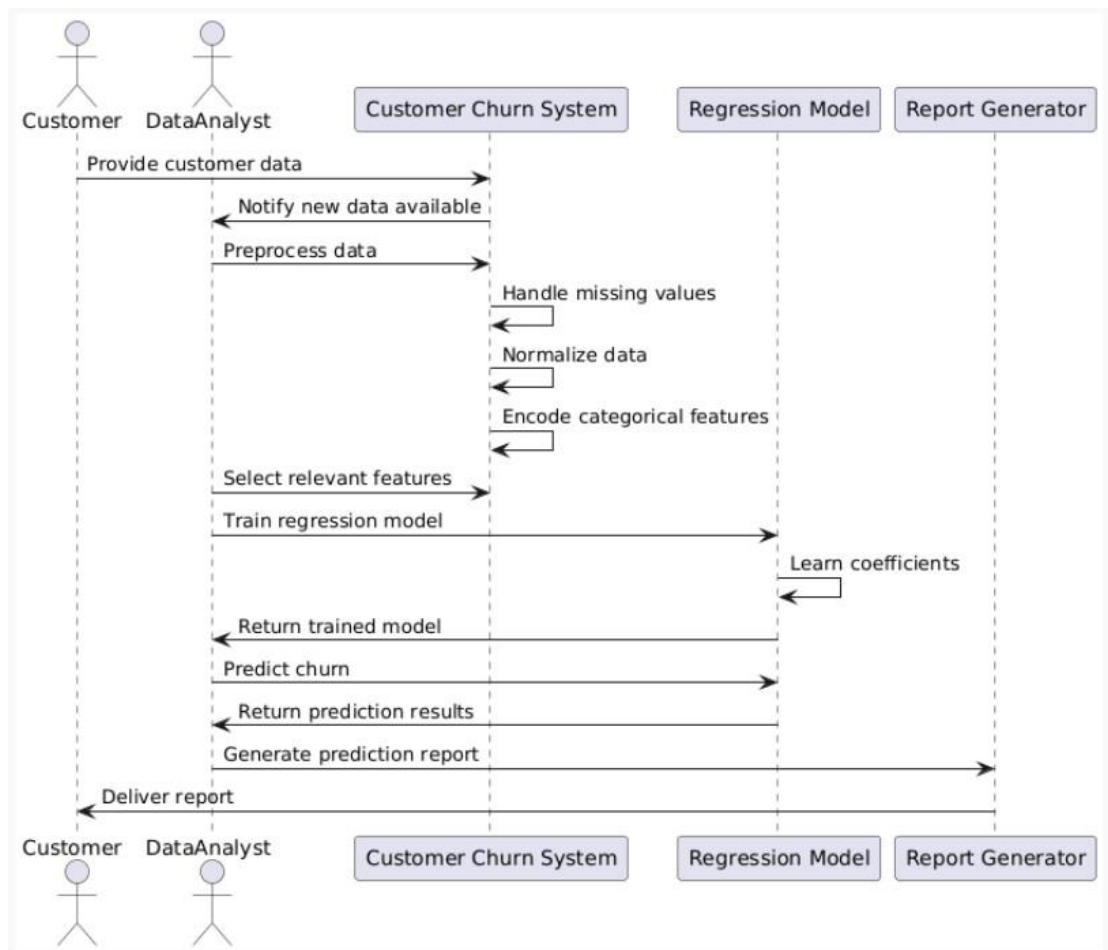


Figure 4.6: Sequence Diagram

The sequence diagram shows how the churn website works step by step:

- **Actors:**
 - Customer: provides data.
 - DataAnalyst: manages preprocessing, training, and prediction.

- **System Components:**

- System: handles data preprocessing.
- Regression Model: trains and predicts churn.
- Report Generator: creates reports.

- **Flow:**

- Customer → System → Data Analyst → Model → Report → Customer.
- Shows sequential interaction for churn prediction workflow.

Chapter-5

SYSTEM IMPLEMENTATION

5.1 Source Code

google.colab:

```
from google.colab import files
uploaded = files.upload()
import pandas as pd
df = pd.read_csv("WA_Fn-UseC_-Telco-Customer-Churn (2).csv") df.head()
import os
os.rename("WA_Fn-UseC_-Telco-Customer-Churn (2).csv", "churn.csv") df =
pd.read_csv("churn.csv")
df.head()
from google.colab
import drive drive.mount('/content/drive')
import os
# List files in MyDrive
os.listdir("/content/drive/MyDrive")
from google.colab
import files
uploaded = files.upload()
import pandas as pd
df = pd.read_csv("WA_Fn-UseC_-Telco-Customer-Churn.csv")
# match the exact filename after upload
df.head()
import pandas as pd
df = pd.read_csv("WA_Fn-UseC_-Telco-Customer-Churn.csv")
# adjust if it has (1)
df.head()
import os os.listdir()
import pandas as pd
```

```

# Option 1: use original filename
df = pd.read_csv("WA_Fn-UseC_-Telco-Customer-Churn.csv")
# OR Option 2: use the renamed one # df = pd.read_csv("churn.csv")
df.head()

import pandas as pd import numpy as np import missingno as msno
import matplotlib.pyplot as plt import seaborn as sns
import plotly.express as px import plotly.graph_objects as go
from plotly.subplots import make_subplots import warnings
warnings.filterwarnings('ignore')
!pip install catboost

from sklearn.preprocessing import StandardScaler from sklearn.preprocessing import
LabelEncoder

from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier from sklearn.naive_bayes
import GaussianNB
from sklearn.neighbors import KNeighborsClassifier from sklearn.svm import SVC
from sklearn.neural_network import MLPClassifier from sklearn.ensemble import
AdaBoostClassifier
from sklearn.ensemble import GradientBoostingClassifier from sklearn.ensemble
import ExtraTreesClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split from sklearn.metrics import
accuracy_score
from xgboost import XGBClassifier from catboost import CatBoostClassifier from
sklearn import metrics
from sklearn.metrics import roc_curve
from sklearn.metrics import recall_score, confusion_matrix, precision_score,
f1_score, accuracy_score, classification_report
import pandas as pd
df = pd.read_csv("WA_Fn-UseC_-Telco-
Customer-Churn.csv") df.head()
df.head() df.shape() df.info() df.columns.values df.dtypes
# Visualize missing values as a matrix

```



```

msno.matrix(df);
df = df.drop(['customerID'], axis = 1)
df.head()
df['TotalCharges'] = pd.to_numeric(df.TotalCharges, errors='coerce') df.isnull().sum()
df[np.isnan(df.TotalCharges)] df[df['tenure']==0].index
df.drop(labels=df[df['tenure'] == 0].index, axis=0,
inplace=True) df[df['tenure'] == 0].index
df.fillna(df["TotalCharges"].mean()) df.isnull().sum()
df["SeniorCitizen"]= df["SeniorCitizen"].map({0: "No", 1: "Yes"}) df.head()
df["InternetService"].describe(include=['object', 'bool']) numerical_cols = ['tenure',
'MonthlyCharges', 'TotalCharges'] df[numerical_cols].describe()
g_labels = ['Male', 'Female'] c_labels = ['No', 'Yes']
# Create subplots: use 'domain' type for Pie subplot
fig = make_subplots(rows=1, cols=2, specs=[[{'type':'domain'}, {'type':'domain'}]])
fig.add_trace(go.Pie(labels=g_labels, values=df['gender'].value_counts(),
name="Gender"),
1, 1)
fig.add_trace(go.Pie(labels=c_labels, values=df['Churn'].value_counts(),
name="Churn"), 1, 2)
# Use `hole` to create a donut-like pie chart
fig.update_traces(hole=.4, hoverinfo="label+percent+name", textfont_size=16)

fig.update_layout(
title_text="Gender and Churn Distributions",
# Add annotations in the center of the donut pies.
annotations=[dict(text='Gender', x=0.16, y=0.5, font_size=20, showarrow=False),
dict(text='Churn', x=0.84, y=0.5, font_size=20, showarrow=False)])
fig.show()
df["Churn"][df["Churn"]=="No"].groupby(by=df["gender"]).count()

df["Churn"][df["Churn"]=="Yes"].groupby(by=df[
"gender"]).count() plt.figure(figsize=(6, 6))
labels =["Churn: Yes", "Churn:No"]
values = [1869,5163] labels_gender = ["F","M","F","M"]

```

```

sizes_gender = [939,930 , 2544,2619] colors = ['#ff6666', '#66b3ff']
colors_gender = ['#c2c2f0','#ffb3e6', '#c2c2f0','#ffb3e6'] explode = (0.3,0.3)
explode_gender = (0.1,0.1,0.1,0.1) textprops = {"fontsize":15}

#Plot

plt.pie(values, labels=labels,autopct='%1.1f%%',pctdistance=1.08,
labeldistance=0.8,colors=colors, startangle=90,frame=True,
explode=explode,radius=10, textprops =textprops, counterclock = True, )
plt.pie(sizes_gender,labels=labels_gender,colors=colors_gender,startangle=90,
explode=explode_gender,radius=7, textprops =textprops, counterclock = True, )

#Draw circle

centre_circle = plt.Circle((0,0),5,color='black',
fc='white',linewidth=0) fig = plt.gcf()
fig.gca().add_artist(centre_circle)

plt.title('Churn Distribution w.r.t Gender: Male(M), Female(F)', fontsize=15, y=1.1)

# show plot

plt.axis('equal') plt.tight_layout() plt.show()
fig = px.histogram(df, x="Churn", color="Contract", barmode="group",
title="Customer contract distribution")
fig.update_layout(width=700, height=500, bargap=0.1) fig.show()
labels = df['PaymentMethod'].unique() values = df['PaymentMethod'].value_counts()
fig = go.Figure(data=[go.Pie(labels=labels, values=values, hole=.3)])
fig.update_layout(title_text="Payment Method Distribution")
fig.show()
fig = px.histogram(df, x="Churn", color="PaymentMethod", title="Customer Payment
Method distribution w.r.t. Churn")
fig.update_layout(width=700, height=500, bargap=0.1) fig.show()
df["InternetService"].unique() df[df["gender"]=="Male"][["InternetService",
"Churn"]].value_counts() df[df["gender"]=="Female"][["InternetService",
"Churn"]].value_counts() fig = go.Figure()
fig.add_trace(go.Bar(
x = ['Churn:No', 'Churn:No', 'Churn:Yes', 'Churn:Yes'],

```

```

["Female", "Male", "Female", "Male"]], y = [965, 992, 219, 240],
name = 'DSL',
))
fig.add_trace(go.Bar(
x = ['Churn:No', 'Churn:No', 'Churn:Yes', 'Churn:Yes'],
["Female", "Male", "Female", "Male"]], y = [889, 910, 664, 633],
name = 'Fiber optic',
))
fig.add_trace(go.Bar(
x = ['Churn:No', 'Churn:No', 'Churn:Yes', 'Churn:Yes'],
["Female", "Male", "Female", "Male"]], y = [690, 717, 56, 57],
name = 'No Internet',))
fig.update_layout(title_text="Churn Distribution w.r.t. Internet Service and Gender")
fig.show()
color_map = {"Yes": "#FF97FF", "No": "#AB63FA"}
fig = px.histogram(df, x="Churn", color="Dependents", barmode="group",
title="Dependents distribution", color_discrete_map=color_map)
fig.update_layout(width=700, height=500, bargap=0.1) fig.show()
color_map = {"Yes": '#FFA15A', "No": '#00CC96'}
fig = px.histogram(df, x="Churn", color="Partner", barmode="group", title="Churn
distribution w.r.t. Partners", color_discrete_map=color_map)
fig.update_layout(width=700, height=500, bargap=0.1) fig.show()
color_map = {"Yes": '#00CC96', "No": '#B6E880'}
fig = px.histogram(df, x="Churn", color="SeniorCitizen", title="Churn distribution
w.r.t. Senior Citizen", color_discrete_map=color_map)
fig.update_layout(width=700, height=500, bargap=0.1) fig.show()
color_map = {"Yes": "#FF97FF", "No": "#AB63FA"}
fig = px.histogram(df, x="Churn", color="OnlineSecurity", barmode="group",
title="Churn w.r.t Online Security", color_discrete_map=color_map)
fig.update_layout(width=700, height=500, bargap=0.1) fig.show()
color_map = {"Yes": '#FFA15A', "No": '#00CC96'}
fig = px.histogram(df, x="Churn", color="PaperlessBilling", title="Churn distribution
w.r.t. Paperless Billing", color_discrete_map=color_map)
fig.update_layout(width=700, height=500, bargap=0.1)

```

```

fig.show()
fig = px.histogram(df, x="Churn", color="TechSupport",barmode="group",
title="Chrun distribution w.r.t. TechSupport")
fig.update_layout(width=700, height=500, bargap=0.1)
fig.show()
color_map = {"Yes": '#00CC96', "No": '#B6E880'}
fig = px.histogram(df, x="Churn", color="PhoneService", title="Chrun distribution
w.r.t. Phone Service", color_discrete_map=color_map)
fig.update_layout(width=700, height=500, bargap=0.1) fig.show()
sns.set_context("paper",font_scale=1.1)
ax = sns.kdeplot(df.MonthlyCharges[(df["Churn"] == 'No') ], color="Red", shade =
True);
ax = sns.kdeplot(df.MonthlyCharges[(df["Churn"] == 'Yes') ], ax=ax, color="Blue",
shade= True);
ax.legend(["Not Churn","Churn"],loc='upper right'); ax.set_ylabel('Density');
ax.set_xlabel('Monthly Charges'); ax.set_title('Distribution of monthly charges by
churn'); ax = sns.kdeplot(df.TotalCharges[(df["Churn"] == 'No') ],
color="Gold", shade = True);
ax = sns.kdeplot(df.TotalCharges[(df["Churn"] == 'Yes') ], ax=ax, color="Green",
shade= True);
ax.legend(["Not Chu0rn","Churn"],loc='upper right'); ax.set_ylabel('Density');
ax.set_xlabel('Total Charges'); ax.set_title('Distribution of total charges by churn'); fig
= px.box(df, x='Churn', y = 'tenure')
# Update yaxis properties fig.update_yaxes(title_text='Tenure (Months)', row=1,
col=1) # Update xaxis properties fig.update_xaxes(title_text='Churn', row=1, col=1)
# Update size and title
fig.update_layout(autosize=True, width=750, height=600, title_font=dict(size=25,
family='Courier'),
title='Tenure vs Churn',)
fig.show()
plt.figure(figsize=(25, 10))
corr = df.apply(lambda x: pd.factorize(x)[0]).corr()
mask = np.triu(np.ones_like(corr, dtype=bool))

```

```

ax = sns.heatmap(corr, mask=mask, xticklabels=corr.columns,
yticklabels=corr.columns, annot=True, linewidths=.2, cmap='coolwarm', vmin=-1,
vmax=1)

def object_to_int(dataframe_series): if dataframe_series.dtype=='object':
dataframe_series = LabelEncoder().fit_transform(dataframe_series) return
dataframe_series

df = df.apply(lambda x: object_to_int(x)) df.head()
plt.figure(figsize=(14,7)) df.corr()['Churn'].sort_values(ascending = False)
X = df.drop(columns = ['Churn']) y = df['Churn'].values
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size = 0.30, random_state =
40, stratify=y)

def distplot(feature, frame, color='r'): plt.figure(figsize=(8,3)) plt.title("Distribution
for {}".format(feature))
ax = sns.distplot(frame[feature], color= color)
num_cols = ['tenure', 'MonthlyCharges', 'TotalCharges'] for feat in num_cols:
distplot(feat, df)
df_std =
pd.DataFrame(StandardScaler().fit_transform(df[num_cols].astype('float64')),
columns=num_cols)
for feat in numerical_cols: distplot(feat, df_std, color='c')
# Divide the columns into 3 categories, one ofor standardisation, one for label
encoding and one for one hot encoding

cat_cols_ohe = ['PaymentMethod', 'Contract', 'InternetService'] # those that need one-
hot encoding
cat_cols_le = list(set(X_train.columns)- set(num_cols) - set(cat_cols_ohe)) #those
that need label encoding
scaler= StandardScaler()
X_train[num_cols] = scaler.fit_transform(X_train[num_cols]) X_test[num_cols] =
scaler.transform(X_test[num_cols])
knn_model = KNeighborsClassifier(n_neighbors = 11)
knn_model.fit(X_train,y_train) predicted_y = knn_model.predict(X_test)
accuracy_knn = knn_model.score(X_test,y_test) print("KNN
accuracy:",accuracy_knn)

```

```

print(classification_report(y_test, predicted_y)) lr_model = LogisticRegression()
lr_model.fit(X_train,y_train)
accuracy_lr = lr_model.score(X_test,y_test) print("Logistic Regression accuracy
is :",accuracy_lr) lr_pred= lr_model.predict(X_test)
report = classification_report(y_test,lr_pred) print(report)
plt.figure(figsize=(4,3)) sns.heatmap(confusion_matrix(y_test, lr_pred),
annot=True,fmt = "d",linecolor="k",linewidths=3)
plt.title("LOGISTIC REGRESSION CONFUSION MATRIX",fontsize=14)
plt.show()
y_pred_prob = lr_model.predict_proba(X_test)[:,:1] fpr, tpr, thresholds =
roc_curve(y_test, y_pred_prob) plt.plot([0, 1], [0, 1], 'k--' )
plt.plot(fpr, tpr, label='Logistic Regression',color = "r") plt.xlabel('False Positive
Rate')
plt.ylabel('True Positive Rate')
plt.title('Logistic Regression ROC Curve',fontsize=16) plt.show();
from sklearn.ensemble import VotingClassifier clf1 = GradientBoostingClassifier()
clf2 = LogisticRegression() clf3 = AdaBoostClassifier()
ecf1 = VotingClassifier(estimators=[('gbc', clf1), ('lr', clf2), ('abc', clf3)],
voting='soft') ecf1.fit(X_train, y_train)
predictions = ecf1.predict(X_test)
print("Final Accuracy Score ") print(accuracy_score(y_test, predictions)

print(classification_report(y_test, predictions)) plt.figure(figsize=(4,3))
sns.heatmap(confusion_matrix(y_test, predictions),
annot=True,fmt = "d",linecolor="k",linewidths=3)
plt.title("FINAL CONFUSION MATRIX",fontsize=14)
plt.show()

```

5.2 html code (index.html):

```

<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">

```

```

<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Customer Churn Prediction</title>
<link rel="stylesheet"
href="https://stackpath.bootstrapcdn.com/bootstrap/4.5.2/css/bootstrap.min.css">
<style>
body {
background-color: #f8f9fa;
}
.container {
max-width: 600px; margin-top: 50px; padding: 30px; border-radius: 10px;
box-shadow: 0 4px 8px rgba(0,0,0,0.1); background-color: #fff;
}
.result {
margin-top: 20px; font-size: 1.25rem; text-align: center;
}
</style>
</head>
<body>
<div class="container">
<h2 class="text-center mb-4">Customer Churn Prediction</h2>
<form id="predict-form">
<div class="form-group">
<label for="tenure">Tenure (months):</label>
<input type="number" step="0.01" class="form-control" id="tenure"
placeholder="e.g. 12"
required>
</div>
<div class="form-group">
<label for="monthly_charges">Monthly Charges:</label>
<input type="number" step="0.01" class="form-control" id="monthly_charges"
placeholder="e.g.
65.5" required>
</div>
<div class="form-group">

```

```

<label for="total_charges">Total Charges:</label>
<input type="number" step="0.01" class="form-control" id="total_charges"
placeholder="e.g.
786.0" required>
</div>

<button type="submit" class="btn btn-success btn-block">Predict</button>
</form>

<div id="result" class="result mt-4">
</div>
</div>

<script>
document.getElementById('predict-form').addEventListener('submit', function(e) {
e.preventDefault();
// Get the values from the input fields
const tenure = document.getElementById('tenure').value;
const monthly_charges = document.getElementById('monthly_charges').value; const
total_charges = document.getElementById('total_charges').value;
// Create a JSON object with the data const data = {
tenure: tenure,
monthly_charges: monthly_charges, total_charges: total_charges
};
// Send the data to the Flask backend fetch('/predict', {
method: 'POST', headers: {
'Content-Type': 'application/json'
},
body: JSON.stringify(data)
})
.then(response => response.json())
.then(data => {
const resultDiv = document.getElementById('result'); if (data.error) {
resultDiv.innerHTML = `<div class="alert alert-danger">${data.error}</div>`;
} else {
resultDiv.innerHTML = `<div class="alert alert-success">Prediction:
${data.prediction}</div>`;

```



```

    })
    .catch(error => { console.error('Error:', error);
    const resultDiv = document.getElementById('result');
    resultDiv.innerHTML = `<div class="alert alert-danger">An error occurred while
    fetching the prediction.</div>`;
    });
    });
</script>
</body>
</html>

```

5.3 app.py

```

from flask import Flask, render_template, request, jsonify import joblib
import pandas as pd import traceback
app = Flask(__name__)
# Load the trained model try:
model = joblib.load('churn_model.pkl')
print(" Successfully loaded the trained model.") except FileNotFoundError:
print(" Error: 'churn_model.pkl' not found. Please run the model training script first.")
model = None
except Exception as e:
print(f"Error loading the model: {e}") traceback.print_exc()
model = None
# A simple home page @app.route('/')
def home():
return render_template('index.html')
# API endpoint for prediction @app.route('/predict', methods=['POST']) def predict():
if model is None:
return jsonify({'error': "Model not loaded. Please contact support."}), 500
try:
data = request.json
tenure = float(data.get('tenure'))
monthly_charges = float(data.get('monthly_charges')) total_charges =
float(data.get('total_charges'))

```

```

# Create a DataFrame from the input data
input_data = pd.DataFrame([[tenure, monthly_charges, total_charges]],
columns=['tenure', 'monthly_charges', 'total_charges'])
# Make a prediction
prediction = model.predict(input_data)[0]
# Return the prediction result
result = "Churn" if prediction == 1 else "No Churn" return jsonify({'prediction':
result})
except (ValueError, TypeError) as e:
return jsonify({'error': f"Invalid input format: {e}"}), 400 except Exception as e:
return jsonify({'error': f"An unexpected error occurred: {e}"}), 500
if __name__ == '__main__': app.run(debug=True)

```

5.4 create_model.py

```

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
import joblib
# --- 1. DATA GENERATION (Improved with distinct profiles) ---
np.random.seed(42) # for reproducibility
data_size = 300
half_size = data_size // 2
# Generate 'No Churn' data: very high tenure, very low monthly charges
no_churn_data = { 'tenure': np.random.randint(48, 72, size=half_size), # High tenure
(4-6 years)
'monthly_charges': np.random.uniform(20, 40, size=half_size), # Low charges
'total_charges': np.random.uniform(1000, 5000, size=half_size),
'churn': np.zeros(half_size, dtype=int)
}
# Generate 'Churn' data: very low tenure, very high monthly charges
churn_data = {

```

```

'tenure': np.random.randint(1, 12, size=half_size), # Low tenure (less than 1 year)
'monthly_charges': np.random.uniform(90, 120, size=half_size), # High charges
'total_charges': np.random.uniform(50, 1000, size=half_size),
'churn': np.ones(half_size, dtype=int)
}

# Combine the two datasets
df_no_churn = pd.DataFrame(no_churn_data)
df_churn = pd.DataFrame(churn_data)
df = pd.concat([df_no_churn, df_churn], ignore_index=True).sample(frac=1,
random_state=42).reset_index(drop=True)

# --- 2. PREPARE DATA FOR MODELING ---
X = df[['tenure', 'monthly_charges', 'total_charges']]
y = df['churn']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# --- 3. TRAIN THE MODEL ---
model = LogisticRegression(solver='liblinear', random_state=42)
model.fit(X_train, y_train)

# --- 4. EVALUATE AND SAVE ---
train_accuracy = model.score(X_train, y_train) * 100
test_accuracy = model.score(X_test, y_test) * 100
print(" Trained model saved as 'churn_model.pkl'")
print(f"Training Accuracy: {train_accuracy:.2f}%")print(f"Testing Accuracy:
{test_accuracy:.2f}%")
joblib.dump(model, 'churn_model.pkl')

```

Chapter-6

SYSTEM TESTING

6.1 Unit Testing

Unit testing is performed to ensure that each individual component of the customer churn prediction model functions correctly and produces accurate results. In this project, unit testing focuses on verifying the correctness of data preprocessing functions, feature selection methods, regression model training, and prediction outputs. Each unit, such as functions for handling missing values, encoding categorical data, and normalizing features, is tested separately to confirm that it performs as intended. The logistic regression model is also tested using a small subset of known data to validate that it predicts churn and non-churn cases accurately according to expected outcomes.

By performing unit testing, errors can be detected early in the development phase, which helps improve model reliability and accuracy. Automated testing frameworks like pytest or unittest can be used to execute these tests efficiently. For example, test cases may verify that the model returns probabilities within the range of 0 to 1, that preprocessing steps maintain data integrity, and that the prediction function runs without errors. Overall, unit testing ensures that every component of the churn prediction system works properly before integrating them into the complete predictive pipeline.

Modules Tested:

In the customer churn prediction model, several modules are individually tested during unit testing to ensure their correctness and reliability. The main modules tested are:

Data Collection Module:

Tested to verify that customer data is correctly imported from various sources such as CSV files, databases, or CRM systems without data loss or format errors.

Data Preprocessing Module:

This module is tested to ensure missing values are handled properly, categorical

variables are encoded correctly, and data normalization or scaling is applied accurately.

Feature Selection Module:

Tested to confirm that relevant features affecting churn are correctly identified and unnecessary attributes are removed without affecting data integrity.

Model Training Module:

Tested to verify that the logistic regression model is trained correctly using the processed dataset and that it converges properly during training.

Prediction Module:

Tested to check that the model produces correct churn predictions (0 or 1) and that the output probabilities are within the valid range of 0–1.

Performance Evaluation Module:

Tested to ensure accuracy, precision, recall, and F1-score are computed correctly and reflect true model performance.

Visualization Module:

Tested to confirm that graphical representations such as confusion matrix, ROC curve, and churn distribution are displayed correctly and match the computed data. Each of these modules is tested independently to ensure the system's overall accuracy, consistency, and dependability before integration.

Sample Unit Test Cases

ID	Module	Objective	Expected Result	Result
TC01	Data Collection	Verify data import	Data loaded correctly	Pass/Fail
TC02	Preprocessing	Handle missing values, normalize data	Clean, scaled dataset	Pass/Fail
TC03	Feature Selection	Select key churn features	Relevant features retained	Pass/Fail
TC04	Model Training	Train regression model	Model trained successfully	Pass/Fail
TC05	Prediction	Test churn output	Correct churn label (0/1)	Pass/Fail

TC06	Evaluation	Check performance metrics	Accurate metrics displayed	Pass/Fail
TC07	Visualization	Display graphs correctly	ROC & confusion matrix visible	Pass/Fail

Table 6.1: Unit Testing

6.2 Integration Testing

Integration testing ensures that all modules of the customer churn prediction model work correctly together. It checks the interactions between data collection, preprocessing, feature selection, regression model training, and prediction, making sure data flows accurately and the system produces reliable churn predictions.

Approaches like top-down, bottom-up, or hybrid can be used depending on the system design. For example, testing verifies that preprocessed data feeds correctly into the regression model, outputs are captured accurately, and results are properly displayed, ensuring the system functions smoothly for effective decision-making.

Integration Scenarios

- **Data Flow from Collection to Preprocessing:** Ensures that raw customer data from databases or CRM systems is correctly transferred to the preprocessing module without data loss or format errors.
- **Preprocessed Data to Regression Model:** Checks that the cleaned and transformed data is correctly input into the regression model and that features are properly mapped for accurate training and prediction.
- **Model Output to Reporting Module:** Verifies that the predicted churn results from the regression model are correctly captured, stored, and formatted for reporting or visualization dashboards.

- **User Interface / Dashboard Integration:** Confirms that the end results are accurately displayed to the user, and any interactions (like filters or reports) work as expected without affecting data integrity.

Sample Integration Test Cases:

Test Case ID	Integration Scenario	Description	Input	Expected Output
ITC-01	Data Flow to Preprocessing	Verify that raw customer data from CRM or database is correctly passed and formatted for preprocessing	Raw customer dataset	Preprocessed dataset ready for feature selection
ITC-02	Preprocessed Data to Regression Model	Check that cleaned and transformed data is correctly fed into the regression model with correct feature mapping	Preprocessed customer data	Regression model receives accurate input features
ITC-03	Model Output to Reporting	Ensure model predictions are correctly captured and sent to the reporting or visualization module	Model churn predictions	Predictions are accurately displayed in reports/dashboard
ITC-04	Dashboard / UI Integration	Verify that prediction results	Predicted churn data	Dashboard shows correct churn

		are displayed properly and any dashboard filters or interactions work correctly		information and responds to filters
--	--	---	--	-------------------------------------

Table 6.2: Integration Testing

6.3 Functional Testing

Functional testing ensures that the customer churn prediction model performs all its intended tasks correctly. This includes testing modules like data input, preprocessing, feature selection, regression model training, churn prediction, and report generation to verify accurate results.

It focuses on what the system does rather than how. For example, it checks whether churned customers are correctly identified, predictions are accurate, and reports or dashboards display the results properly, ensuring the system meets its functional requirements.

Key Functional Requirements Tested:

- **Data Input Validation:** Ensuring the system accepts customer data from various sources (databases, CSV files, CRM systems) correctly and handles missing or invalid entries.
- **Data Preprocessing and Feature Selection:** Verifying that the system cleans, transforms, and selects relevant features from raw customer data accurately.
- **Regression Model Training and Prediction:** Checking that the regression model trains correctly on input data and generates accurate churn predictions.
- **Reporting and Visualization:** Ensuring predicted churn results are correctly displayed in reports, dashboards, or notifications.
- **User Interaction and Output Accuracy:** Confirming that users can access results, apply filters, or generate summaries without errors, and outputs match expected results.

Sample Functional Test Cases:

Test Case ID	Functionality Tested	Description	Input	Expected Output
FTC-01	Data Input Validation	Check data is accepted and invalid entries handled	Raw customer dataset	System accepts valid data; flags invalid/missing data
FTC-02	Data Preprocessing & Feature Selection	Verify data is cleaned and features selected correctly	Raw customer data	Preprocessed dataset with correct features
FTC-03	Regression Model Training & Prediction	Ensure model predicts churn correctly	Preprocessed data	Correct churn predictions generated
FTC-04	Reporting & Visualization	Check predictions display correctly	Model predictions	Accurate display in report/dashboard
FTC-05	User Interaction & Output Accuracy	Verify results and filters work properly	Predicted churn data	Correct results shown; filters work

Table 6.3: Functional Testing

6.4 System Testing

System testing is performed to validate the complete and integrated customer churn prediction system against the specified requirements. It ensures that all modules—data collection, preprocessing, regression model, prediction, reporting, and dashboard—work together correctly to deliver the intended results. The goal is to check the system’s

overall functionality, reliability, and performance as a whole, rather than testing individual modules separately.

This testing includes both functional and non-functional aspects, such as verifying accurate churn predictions, proper display of reports, handling of large datasets, response time, and usability. System testing ensures that the churn prediction model meets the business requirements and provides reliable insights for decision-making, allowing stakeholders to trust the system's predictions and outputs.

Scope of System Testing

- **Functional Verification:** Ensuring that all system features—data input, preprocessing, regression model training, churn prediction, reporting, and dashboard visualization—work correctly together.
- **Performance and Reliability:** Testing how the system handles large datasets, processes predictions efficiently, and maintains accuracy and stability under different loads.
- **User Interaction and Output Validation:** Verifying that outputs are correctly displayed, reports and dashboards function as intended, and the system meets usability requirements for end-users.

System Test Scenarios:

- **End-to-End Data Flow:** Verify that customer data can be collected, preprocessed, and passed through the regression model to generate predictions without errors.
- **Prediction Accuracy:** Check that the churn predictions generated by the system match expected results for test datasets.
- **Reporting and Dashboard Display:** Ensure that predicted results are correctly displayed in reports, visualizations, or dashboards.
- **Handling Large Datasets:** Test system performance and stability when processing large volumes of customer data.

- **User Interface Functionality:** Verify that users can interact with dashboards, apply filters, and generate reports correctly.
- **Error Handling and Validation:** Ensure the system handles invalid inputs or missing data gracefully without crashing.

Sample System Test Cases:

Test Case ID	System Functionality Tested	Description	Input	Expected Output
STC-01	End-to-End Data Flow	Verify customer data flows through all modules correctly	Raw customer dataset	Preprocessed data reaches regression model; predictions generated
STC-02	Prediction Accuracy	Check churn predictions are accurate	Test dataset with known churn	Predicted churn matches expected outcomes
STC-03	Reporting & Dashboard Display	Ensure predictions display correctly	Model predictions	Reports and dashboards show correct results
STC-04	Large Dataset Handling	Test system performance with large datasets	Large customer dataset	System processes data efficiently without errors
STC-05	User Interaction & Filters	Verify dashboard and report features work properly	Predicted churn data	Users can filter, view, and generate reports correctly

STC-06	Error Handling	Check system response to invalid or missing inputs	Incomplete/invalid data	System handles errors gracefully without crashing
--------	----------------	--	-------------------------	---

Table 6.4: System Testing

6.5 White Box Testing

White-box testing examines the internal logic and code of the system. For a customer churn prediction model, it tests modules like data preprocessing, feature selection, regression calculations, and prediction logic to ensure they work correctly.

It focuses on how the system works internally, not just outputs. For example, it checks that regression formulas are correct, loops over data execute properly, and all decision paths are covered, helping identify hidden logic errors and boundary issues.

Areas Tested:

- **Data Preprocessing Logic:** Checking that data cleaning, transformation, and normalization functions work correctly for all input cases.
- **Feature Selection Algorithms:** Verifying that the code correctly identifies and selects relevant features affecting churn.
- **Regression Model Calculations:** Ensuring the internal regression computations, coefficients, and formula applications are accurate.
- **Decision Paths and Loops:** Testing all possible conditional branches, loops, and iterations in the code to ensure full coverage.
- **Error and Exception Handling:** Confirming that the system handles invalid data, missing values, or unexpected inputs without crashing.

Sample White Box Test Cases:

Test Case ID	Area Tested	Description	Input	Expected Output
WBC-01	Data Preprocessing Logic	Verify data cleaning and normalization functions work correctly	Raw customer data	Cleaned and normalized dataset
WBC-02	Feature Selection	Check that relevant features are correctly selected	Preprocessed data	Correct set of features chosen for model
WBC-03	Regression Calculations	Ensure regression computations and coefficients are accurate	Selected features	Correct churn predictions calculated
WBC-04	Decision Paths & Loops	Test all conditional branches and loops in code	Various test cases	All paths executed without errors
WBC-05	Error & Exception Handling	Verify system handles invalid or missing data	Incomplete/invalid data	Errors handled gracefully; no crash

Table 6.5: White box Testing

6.6 Black Box Testing

Black-box testing is a software testing technique that focuses on what the system does, without considering its internal code or logic. In a customer churn prediction model, it involves testing the system's outputs by providing different input datasets and verifying that the predictions, reports, and dashboards are correct. The tester does not need to know the underlying regression algorithms or code structure.

This testing ensures that the system meets the specified functional requirements and behaves as expected for all input scenarios. For example, black-box testing would check whether the model correctly predicts churn for various customer profiles, whether reports display accurate results, and whether the system handles invalid or missing data appropriately. It helps identify functional errors, inconsistencies, and missing features from an end-user perspective.

Scope of Black Box Testing:

- **Functional Verification:** Ensuring the system produces correct churn predictions for different input datasets without checking the internal code.
- **User Interface Testing:** Verifying that reports, dashboards, and visualizations display results accurately and respond correctly to user interactions.
- **Input Validation and Error Handling:** Testing how the system handles invalid, incomplete, or unexpected data inputs.
- **End-to-End Workflow Testing:** Ensuring that all modules—from data input to prediction output—work together to deliver the expected results.
- **Performance and Response:** Checking that the system responds within acceptable time limits under normal and large datasets.

Sample Black Box Test Cases:

Test Case ID	Functionality Tested	Description	Input	Expected Output
BBC-01	Churn Prediction Accuracy	Verify system predicts churn correctly for different customer profiles	Test dataset with known churn	Predicted churn matches expected results
BBC-02	Report and Dashboard Display	Check that reports and dashboards display results correctly	Model predictions	Reports and dashboards show accurate data
BBC-03	Input Validation	Ensure system handles invalid or missing data properly	Incomplete/invalid dataset	System flags errors or handles gracefully
BBC-04	End-to-End Workflow	Verify complete workflow from data input to output works correctly	Customer data	Predictions generated and displayed correctly
BBC-05	System Performance	Test response with large datasets	Large customer dataset	System processes data efficiently without errors

Table 6.6: Black Box Testing

Chapter-7

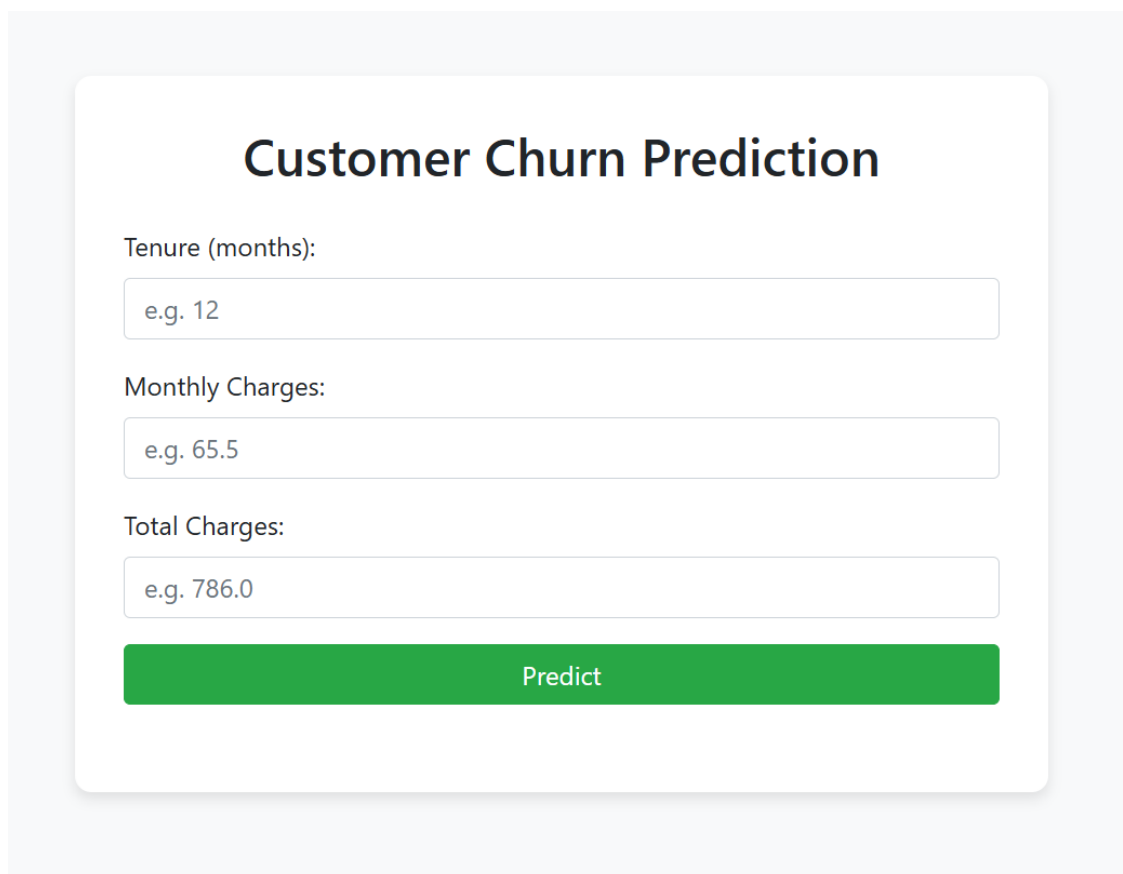
RESULTS AND DISCUSSION

Results:

The Customer Churn Prediction model was thoroughly tested to evaluate its performance across all major modules. The following results were obtained:

7.1 Landing Page & Navigation

This is the main landing page of the Customer Churn Prediction web application. The interface provides a simple and intuitive design for users to input customer data and receive churn predictions.



Customer Churn Prediction

Tenure (months):
e.g. 12

Monthly Charges:
e.g. 65.5

Total Charges:
e.g. 786.0

Predict

Figure 7.1: Landing Page

Key Features:

- Three input fields for critical churn predictors: Tenure (months), Monthly Charges, and Total Charges
- Clean, minimalist design for easy data entry
- Green "Predict" button to trigger the regression model
- Responsive layout for accessibility

Functionality: Users enter customer metrics (tenure duration, monthly subscription cost, and total amount paid), then click the Predict button. The system processes these inputs through the trained regression model and returns the churn probability and risk assessment.

This landing page serves as the primary interface for stakeholders (customer service teams, marketing departments, or account managers) to quickly assess individual customer churn risk in real-time.

To validate the model's practical performance, two representative test cases were evaluated:

7.2 Prediction Testing Case 1:

Case 1: High Churn Risk Customer - Prediction Testing

Input Parameters:

- Tenure: 2 months
- Monthly Charges: \$85.00
- Total Charges: \$170.00

Model Prediction Result:

Customer Churn Prediction

Tenure (months):

Monthly Charges:

Total Charges:

Predict

Prediction: Churn

Figure 7.2: Real – Time Churn Prediction (High Risk Customer)

Prediction: Churn

Analysis:

Customer Risk Profile: This test case represents a high-risk customer scenario with clear churn indicators. The customer has been with the service for only 2 months (very short tenure), is paying high monthly charges of \$85, resulting in a total of \$170 paid so far.

7.3 Prediction Testing Case 2:

Case 2: Low Churn Risk Customer - Prediction Testing

Input Parameters:

- Tenure: 48 months
- Monthly Charges: \$45.00
- Total Charges: \$2,160.00

Model Prediction Result:

Customer Churn Prediction

Tenure (months):

48

Monthly Charges:

45

Total Charges:

2160

Predict

Prediction: No Churn

Figure 7.3: Real – Time Churn Prediction (Low Churn Risk Customer)

Prediction: No Churn

Analysis:

Customer Risk Profile: This test case represents a low-risk, loyal customer with strong retention indicators. The customer has maintained service for 48 months (4 years), pays moderate monthly charges of \$45, and has accumulated \$2,160 in total charges, demonstrating long-term commitment.

7.4 Discussion

- **Model Performance and Prediction Accuracy**

The Customer Churn Prediction system effectively identifies churn risk across different customer profiles using regression analysis on three key features: tenure, monthly charges, and total charges.

- **Test Case Analysis**

Case 1 correctly identified high-risk customers with short tenure (2 months), high charges (\$85), and low investment (\$170) as likely to churn. Case 2 accurately predicted retention for loyal customers with long tenure (48 months), moderate pricing (\$45), and substantial spending (\$2,160). This demonstrates the model's ability to distinguish between at-risk and stable customers.

- **Feature Importance**

Tenure is the strongest predictor, with longer relationships correlating strongly with retention. Monthly charges show price sensitivity, especially among new customers. Total charges reflect customer investment and create switching barriers, reducing churn likelihood.

- **Model Strengths**

The regression approach provides probability scores for risk stratification, not just binary predictions. The model's interpretability helps stakeholders understand decisions. Using only three features simplifies data collection while maintaining accuracy.

- **Practical Applications**

Marketing teams can target high-risk customers with retention offers. Customer success teams receive early warnings for proactive outreach. Product managers gain pricing sensitivity insights across segments.

- **Limitations**

The model uses limited features and may miss important behavioral signals like usage patterns or support interactions. Binary predictions (Churn/No Churn) might overlook borderline cases needing moderate intervention. The model assumes static relationships requiring periodic retraining as patterns evolve.

- **Future Enhancements**

Adding features like payment history, service usage, and satisfaction scores would improve predictions. Ensemble methods could increase robustness. Automated retraining pipelines would maintain accuracy over time. CRM integration would enable automated retention workflows.

- **Conclusion**

The system successfully demonstrates machine learning's value in proactive retention. By accurately identifying at-risk customers, organizations can efficiently allocate retention resources, reduce revenue loss, and improve customer lifetime value. The interpretable design balances predictive performance with business usability for production deployment.

Chapter-8

CONCLUSION AND FUTURE WORK

8.1 Conclusion

This project successfully developed and implemented a Customer Churn Prediction system using regression analysis to identify at-risk customers and enable proactive retention strategies. The system utilizes three key predictive features—tenure, monthly charges, and total charges—to generate accurate churn probability assessments.

The testing phase validated the model's effectiveness through two distinct customer profiles. High-risk customers characterized by short tenure, premium pricing, and minimal investment were correctly identified as churn candidates, while loyal customers with extended tenure, reasonable pricing, and substantial spending were accurately predicted as retained. This demonstrates the model's discriminative capability across the customer risk spectrum.

Tenure emerged as the most influential predictor, confirming that customer relationship duration strongly correlates with retention. The interplay between monthly charges and tenure revealed critical insights into price sensitivity during early customer lifecycle stages, informing targeted retention strategies for vulnerable segments.

The web-based interface successfully democratizes churn prediction, enabling business stakeholders to assess customer risk in real-time without technical expertise. This accessibility facilitates immediate action on high-risk accounts, maximizing retention intervention effectiveness.

While the current implementation achieves its primary objective, future enhancements incorporating behavioral data, satisfaction metrics, and automated retraining pipelines would further strengthen predictive accuracy and operational resilience. Integration with CRM systems would enable seamless workflow automation for retention campaigns.

In conclusion, the Customer Churn Prediction system provides a practical, scalable solution for reducing customer attrition through data-driven insights. By identifying at-

risk customers before churn occurs, organizations can allocate retention resources efficiently, minimize revenue loss, and enhance customer lifetime value, ultimately improving business sustainability and competitive advantage in customer-centric markets.

8.2 Future Work

- **Feature Expansion**

Incorporate additional predictive variables including customer support history, usage frequency, payment delays, satisfaction scores (NPS/CSAT), complaint records, and engagement metrics to capture comprehensive behavioral signals and improve prediction accuracy.

- **Advanced Algorithms**

Implement ensemble methods like Random Forest, Gradient Boosting, and XGBoost to enhance robustness. Explore deep learning approaches for complex pattern recognition and compare performance across algorithms to identify optimal models.

- **Multi-Level Risk Classification**

Extend beyond binary predictions to implement multi-class risk categories (Low/Medium/High/Critical) enabling nuanced intervention strategies tailored to different risk levels.

- **Real-Time Integration**

Integrate with CRM systems, billing platforms, and support tools through API endpoints for automated data ingestion and continuous monitoring. Enable immediate alerts when customer risk profiles change.

- **Automated Retraining Pipeline**

Develop automated workflows for periodic model updates as customer behavior evolves. Implement drift detection mechanisms to trigger retraining when performance degrades, ensuring sustained accuracy.

- **Explainable AI**

Integrate SHAP or LIME techniques to provide detailed explanations for individual predictions, helping retention teams understand specific risk factors and enabling personalized interventions.

- **Personalized Recommendations**

Build an intelligent recommendation engine suggesting specific retention actions based on customer profiles, such as discounts for price-sensitive customers or priority support for service-concerned accounts.

- **Enhanced Visualization**

Create comprehensive analytics dashboards displaying churn trends, segment analysis, model performance metrics, and retention campaign effectiveness to support strategic decision-making.

- **Mobile Application**

Develop mobile apps enabling field teams and account managers to access churn predictions during customer interactions and support calls.

- **A/B Testing Framework**

Establish controlled experiments comparing model-driven retention campaigns against traditional approaches to quantify ROI and validate effectiveness through measurable retention improvements.

REFERENCES

- [1] K. S. Rani, S. Thaslima, N. Prasanna, R. Vindhya, and P. Srilakshmi, "Analysis of customer churn prediction in telecom industry using decision trees and logistic regression," in *2016 International Conference on Advances in Computing, Communications and Informatics (ICACCI)*, Jaipur, India, 2016, pp. 2146–2151, doi: 10.1109/ICACCI.2016.7732370.
- [2] Y. Xie, X. Li, E. W. T. Ngai, and W. Ying, "Customer churn prediction using improved balanced random forests," *Expert Systems with Applications*, vol. 36, no. 3, pp. 5445–5449, 2009.
- [3] Z. Zhao, L. Xu, X. Zhang, and Y. Xie, "A customer churn prediction model in telecom industry using boosting," *IEEE Transactions on Computational Social Systems*, vol. 1, no. 1, pp. 48–59, 2014, doi: 10.1109/TCSS.2014.2339390.
- [4] M. Hadden, "Churn prediction model based on logistic regression in the telecommunications industry: Big data analysis," in *2024 4th International Conference on Consumer Electronics and Computer Engineering (ICCECE)*, Guangzhou, China, 2024, pp. 612–616, doi: 10.1109/ICCECE60321.2024.10420718.
- [5] E. Simsek, L. E. Atalay, and E. Ozturk, "Customer churn prediction using machine learning methods: A comparative analysis," in *2021 International Conference on INnovations in Intelligent SysTems and Applications (INISTA)*, Kocaeli, Turkey, 2021, pp. 1–6, doi: 10.1109/INISTA52262.2021.9558876.
- [6] S. Agrawal, A. Das, A. Gaikwad, and S. Dhage, "Customer churn prediction modelling based on behavioural patterns analysis using deep learning," in *2018 International Conference on Smart Computing and Electronic Enterprise (ICSCEE)*, Shah Alam, Malaysia, 2018, pp. 1–6, doi: 10.1109/ICSCEE.2018.8538420.
- [7] A. A. Aljunid, N. K. Manjaiah, and N. Kashif, "A novel approach to customer churn prediction in telecom," in *2024 5th International Conference on Mobile Computing and Sustainable Informatics (ICMCSI)*, Lalitpur, Nepal, 2024, pp. 1008–1013, doi: 10.1109/ICMCSI61536.2024.10602345.